

MODULE 13 · DOMAIN 3 - DATA TRANSFORMATION AND MODELING

Gold layer objects

Tables, views, materialised views and streaming tables — what each stores, when each computes, and how to choose.

DOMAIN 3 of 7 EXAM WEIGHT 22% MODULE 13 of 36

The four objects differ on two axes: **is the result stored**, and **when is it computed**.
Everything else follows.

The four objects

OBJECT	STORES	COMPUTED	FRESHNESS
Table	Rows	When written	As of the last write
View	Only the query	At every query	Always current
Materialised view	The query result	At refresh	As of the last refresh
Streaming table	Rows, appended	As data arrives	Continuously, or per refresh

ON THE EXAM

Read the last two columns together. A view is always current and always pays for it. A materialised view is fast and can be stale.

Table

- ◆ **The default and the simplest.** Rows written by a job, read by anyone.
- ◆ **Refresh is your responsibility**, through whatever job builds it.
- ◆ **Full control over the write pattern:** overwrite, merge, or append.
- ◆ **Choose it when the build logic is complex**, or when the write needs to be a MERGE.

ON THE EXAM

A gold table built by a scheduled job is not old-fashioned. It is the right answer whenever you need control over how rows are written.

View

- ◆ **Stores a query, not data.** Zero storage cost.
- ◆ **Always current**, because it reads the base tables at query time.
- ◆ **Every reader pays the full cost.** A ninety-second query takes ninety seconds for everybody, every time.
- ◆ **Good for lightweight logic:** a filter, a rename, a simple join, or hiding columns.

ON THE EXAM

A view is also a governance tool. It can expose a subset of a table to a group who have no access to the table itself.

A materialised view stores the **results** of a query and updates them as the sources change, either on a schedule or automatically.

What a materialised view needs

- ◆ **Backed by a serverless pipeline.** The workspace must support serverless pipelines.
- ◆ **A Unity Catalog-enabled pro or serverless SQL warehouse.** Classic will not do.
- ◆ **You must be in the workspace that created it** to refresh it.
- ◆ **Privileges:** SELECT on base tables, USE CATALOG and USE SCHEMA on both source and target, plus CREATE MATERIALIZED VIEW.

ON THE EXAM

Creating a materialised view in Databricks SQL silently creates a serverless pipeline to maintain it. That is where the refresh cost appears.

Refreshing

STATEMENT	EFFECT
REFRESH MATERIALIZED VIEW mv	Refresh, incrementally where possible
REFRESH MATERIALIZED VIEW mv FULL	Force a complete recomputation
REFRESH ... ASYNC	Return immediately rather than blocking
A SCHEDULE clause	Refresh automatically on a cron schedule

ON THE EXAM

REFRESH is synchronous by default, so the statement blocks until the refresh finishes. ASYNC returns straight away.

Incremental refresh

- ◆ **Detects changes in the sources** and recomputes only the affected results, rather than the whole query.
- ◆ **Requires serverless compute.**
- ◆ **From Delta tables, the source tables must have row tracking enabled.** Without it, refreshes are full.
- ◆ **Not every query can be refreshed incrementally.** Where it cannot, Spark falls back to a full recomputation.

ON THE EXAM

Row tracking on the sources is the requirement people miss. A materialised view that silently recomputes in full is usually missing it.

What supports incremental refresh

- ◆ **Delta tables**, both Unity Catalog managed and external.
- ◆ **Other materialised views.**
- ◆ **Streaming tables**, including the targets of AUTO CDC INTO operations.
- ◆ **Unity Catalog managed Iceberg tables**, v2 and v3.

ON THE EXAM

Sources outside this list force a full refresh every time, whatever else you configure.

A streaming table processes each input record **exactly once**. That is the property that decides when you need one.

Do not define a materialised view against a source if records **should only be processed once**. Use a streaming table instead.

Why that rule exists

- ◆ **A materialised view may recompute its query from scratch.** That means re-reading the whole source.
- ◆ **If the source no longer holds the history,** a full recompute silently produces a smaller answer.
- ◆ **A streaming table never re-reads.** It appends what is new and remembers, via a checkpoint.
- ◆ **So streaming tables suit ingestion,** and materialised views suit aggregation over data that persists.

ON THE EXAM

Kafka retains messages for days. Rebuild a materialised view over it after the retention window and the older data is simply gone.

Materialised view or streaming table?

MATERIALIZED VIEW

A stored answer

- Result of an aggregation or join, kept up to date
- Can be recomputed from source at any time
- Refreshed on a schedule or automatically
- Wrong when the source does not retain history

STREAMING TABLE

An append-only accumulation

- Each source record processed exactly once
- Never re-reads what it has already consumed
- Suits ingestion from files or message buses
- Wrong when you need a full aggregate recomputed

THE TIEBREAKER

Ask whether the source can be read again. If it cannot, or should not be, that is a streaming table.

Materialised view limitations

- ◆ No identity columns or surrogate keys.
- ◆ Refreshable only from the workspace that created it.
- ◆ Source tables must be registered in Unity Catalog as managed or external Delta tables.
- ◆ Queryable from a SQL warehouse, Lakeflow pipelines, standard access mode, or dedicated on DBR 15.4 and above.

ON THE EXAM

The identity column limitation matters for dimensional models, where surrogate keys are the normal design.

All four are Unity Catalog objects

- ◆ All appear in the **three-level namespace** and can be granted with GRANT SELECT.
- ◆ Lineage works across all of them, so you can trace a dashboard back to bronze.
- ◆ A materialised view is a **Unity Catalog managed table** that happens to be maintained for you.
- ◆ So the governance answer is the same for all four. The choice is about cost and freshness.

ON THE EXAM

That simplifies exam questions: if a scenario's constraint is governance, it usually does not discriminate between these four.

Choosing for a BI team

IF THE REQUIREMENT IS	CHOOSE
Light logic, must always be current	View
Expensive query, many readers, minutes of staleness acceptable	Materialised view
Complex multi-step build, or MERGE on write	Table, built by a job
Ingesting from files or a message bus	Streaming table
The source does not retain history	Streaming table
Surrogate keys in a dimensional model	Table, not a materialised view

ON THE EXAM

Six lines that answer the objective directly. Note that two different requirements both point at a streaming table.

Tiebreakers from this module

WHEN A QUESTION SAYS

The dashboard is slow for every user

The figures must never be stale

Refreshes cost far more than expected

Records must be processed exactly once

The source is Kafka, or Auto Loader ingest

Classic SQL warehouse, creating an MV

Needs an identity column

REACH FOR

A materialised view, not a view

A view, which computes at query time

Row tracking may not be enabled on the sources

A streaming table

A streaming table, never a materialised view

Not supported. Pro or serverless only.

A table. Materialised views do not support them.

ON THE EXAM

Seven lines. The third is the one that shows up as an unexplained cost rather than an error.

A finance dashboard is backed by a view that joins four silver tables and aggregates 900 million rows. It takes about 80 seconds to load. Around 60 analysts open it through the morning, and each one waits.

The underlying tables are updated once, by a pipeline that finishes at 05:30. Analysts arrive from 08:00.

What should the team do?

- ☐ A Increase the SQL warehouse size so the view query runs faster
- ☐ B Replace the view with a materialised view, refreshed after the 05:30 pipeline
- ☐ C Replace the view with a streaming table
- ☐ D Add result caching so the second and later analysts get a cached result

◆ PAUSE AND DECIDE

What should the team do?

A Increase the warehouse size

Makes 80 seconds into perhaps 40, for every analyst, at permanently higher cost. It does not stop the work being repeated 60 times.

C A streaming table

Streaming tables are for incremental ingestion, not for serving an aggregate. The sources are silver tables updated in batch.

B Materialised view refreshed after the pipeline

Correct. The data changes once a day, so compute once and let 60 analysts read a stored result. This is the documented use case.

D Result caching

Genuinely helps the second reader onwards, and it is fragile: it expires, and any variation in the query misses the cache.

THE SIGNAL

The data changes once a day and the query runs sixty times. That ratio is the whole argument for materialising.

A team creates a materialised view aggregating a large silver table. They expected refreshes to be cheap, since only about 0.3% of the source rows change each day.

The serverless pipeline bill shows each refresh consuming roughly the same compute as a complete rebuild. Refreshes are scheduled hourly.

What is the most likely cause?

- ☐ A The materialised view is being refreshed with the FULL keyword
- ☐ B Row tracking is not enabled on the source table, so incremental refresh is not possible
- ☐ C Hourly refreshes are too frequent and should be reduced to daily
- ☐ D Materialised views always fully recompute; incremental refresh applies only to streaming tables

◆ PAUSE AND DECIDE

What is the most likely cause?

- A

 Refreshing with FULL
Possible, and worth checking. But a scheduled refresh does not use FULL by default, so it would require someone to have written it deliberately.
- C

 Reduce refresh frequency
Reduces how often you pay, not what each refresh costs. It also makes the data staler without fixing anything.
- B

 Row tracking is not enabled on the source
Correct. Incremental refresh from Delta tables requires row tracking on the sources. Without it, Spark cannot identify changed rows and falls back to a full recompute.
- D

 MVs always fully recompute
False. Incremental refresh is a documented materialised view feature; it simply has requirements.

THE SIGNAL

The fallback is silent. An MV that cannot refresh incrementally does not fail — it recomputes fully and sends you the bill.

A team ingests clickstream events from a Kafka topic with seven days of retention. They want a gold object holding a running daily count of events per product, going back two years.

An engineer proposes a materialised view defined directly over the Kafka source, refreshed hourly.

What is wrong with the proposal?

- ☐ A Nothing; a materialised view refreshed hourly is appropriate here
- ☐ B A materialised view may recompute from source, and Kafka only retains seven days, so history would be lost
- ☐ C Materialised views cannot aggregate, only filter and join
- ☐ D Hourly refresh is too slow for clickstream data; it needs continuous processing

◆ PAUSE AND DECIDE

What is wrong with the proposal?

- A

 Nothing is wrong
It would appear to work for a week, then silently lose everything older than the retention window on the first full recompute.
- B

 MV may recompute; Kafka retains only seven days
Correct. Databricks documents this directly: do not define a materialised view against a source where records should only be processed once. Kafka is the named example.
- C

 MVs cannot aggregate
False. Aggregation is one of the main reasons to use one.
- D

 Hourly is too slow
Latency was never stated as a requirement. The problem is data loss, not freshness.

THE SIGNAL

Ingest with a streaming table, then build the materialised view on that table. The streaming table becomes the durable history.

An operations team relies on a gold object showing open incidents. It was built as a materialised view six months ago. Overnight, a major incident is raised, and at 09:00 the dashboard still shows the previous day's figures.

Investigation shows the materialised view was created without a SCHEDULE clause and has not been refreshed since the day it was created.

What is the correct conclusion?

- ☐ A Materialised views refresh automatically; something is broken and should be raised with support
- ☐ B A materialised view is as fresh as its last refresh; it needs a SCHEDULE, or a view should be used if it must always be current
- ☐ C The materialised view should be converted to a streaming table so it updates continuously
- ☐ D The dashboard is caching results and needs its cache cleared

◆ PAUSE AND DECIDE

What is the correct conclusion?

- A

 They refresh automatically; raise it with support
They can be refreshed on a schedule or automatically, but that must be configured. Nothing is broken; nothing was ever scheduled.
- B

 As fresh as its last refresh; add a SCHEDULE, or use a view
Correct. It names the property, gives the fix, and offers the alternative when 'always current' is the actual requirement.
- C

 Convert to a streaming table
Streaming tables are append-only. An open-incidents figure needs incidents to leave the set when closed, which appending cannot express.
- D

 Clear the dashboard cache
The stored result really is a day old. Clearing a cache would re-read the same stale materialised view.

THE SIGNAL

'As fresh as the last refresh' is the defining property. Six months without a refresh is the definition working, not failing.

Is the result stored, and when is it computed? **Then ask whether the source can be read again.** Those two questions choose all four objects.